

DEEP LEARNING FOR FINANCE

MODULE 5 | LESSON 4

LSTMs FOR PREDICTING BTC PRICES

Reading Time	40 minutes
Prior Knowledge	LSTM, Backtesting, Market timing
Keywords	Bidirectional LSTM, BTC, LSTM

1. Introduction

In this lesson, we will deal with bidirectional neural networks using as a benchmark the same setting for Lesson 2.

The idea here is to check whether, under the current architecture, bidirectional LSTMs bring any advantage over the unidirectional ones studied in lesson 2 of this module. Note that the answer to this question, as usual, cannot be extrapolated to other datasets, other architectures, or other time periods. Hence, we recommend that you play around with the notebook once you understand how to use bidirectional LSTMs to see whether you can enhance the performance of the model.

Bidirectional LSTMs have been recently used to try and predict financial market outcomes such as stock prices. Different from returns themselves, prices may follow a somewhat "logical" sequence (e.g., ascending or descending trend), so using bidirectional networks to try and predict these trends seems like a worthwhile and interesting exercise.

You can find a lot of online resources on bidirectional networks for stock price predictions. Some of them simply used the past series of prices (as we will do here), but others incorporate other types of relevant information. Here, we share some of them, in case you want to take a look:

- Althelaya, Khaled, A., et al. "Evaluation of Bidirectional LSTM for Short-and Long-Term Stock Market Prediction." 9th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 2018, pp. 151-156, doi: 10.1109/IACS.2018.8355458. <https://ieeexplore.ieee.org/abstract/document/8355458>
- Liu, Bingchun, et al. "Prediction of SSE Shanghai Enterprises Index Based on Bidirectional LSTM Model of Air Pollutants." *Expert Systems with Applications*, vol.

204, 2022.

<https://www.sciencedirect.com/science/article/abs/pii/S0957417422009113>

- Yang, Mo, and Jing Wang. "Adaptability of Financial Time Series Prediction Based on Bilstm." *Procedia Computer Science*, vol. 199, 2022, pp. 18-25.

<https://www.sciencedirect.com/science/article/pii/S1877050922000035>

So, let's get going!

2. Data

The data that we will use during this lesson is exactly the same as for lesson 2. Hence, to keep things short, we redirect you to the notebook in lesson 2 to see the details of the data-gathering process, as well as some Exploratory Data Analysis (EDA).

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from keras.layers import LSTM, Bidirectional, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
```

```
In [ ]: drivepath = "Kraken_BTCUSD_60_3Q2022.csv"
```

```
In [ ]: df = pd.read_csv(drivepath)

# Given the format of the data, we need to also
df["unix"] = pd.to_numeric(df["unix"])
df["date"] = pd.to_datetime(df["unix"], unit="s")
```

Once we have loaded and formatted our data, we will only work with a few variables:

```
In [ ]: df = df[["date", "close", "volume", "trades"]]
```

```
In [ ]: df.tail()
```

As usual, we calculate the returns for each our to, later on, get the cumulative returns for the different periods and replace the missing values for zeroes given what we know about the data set (**remember** it is not correct to blindly replace missing data for 0; we do it here simply because we do know the reason for that missing data):

```
In [ ]: df["Ret"] = df["close"].pct_change()
df["year"] = df["date"].dt.year
df["Ret"] = df["Ret"].fillna(0)
df["volume"] = df["volume"].fillna(0)
df["trades"] = df["trades"].fillna(0)
df.head()
```

And, finally, for the sake of computational time, we will restrict the sample to post-2021:

```
In [ ]: del df["close"]
df = df.loc[(df["year"] >= 2021)] # To reduce the computational time
```

```
df.head()
```

3. Forecasting

Let's now work on our bidirectional LSTM framework!

As always, let's build our different input/output variables. For these, we will take a very similar approach to what we did in lesson 2, where different cumulative returns for past periods were used to predict the 25-hour ahead return:

```
In [ ]: df = df.reindex(
        columns=[
            "date",
            "Ret",
            "volume",
            "trades",
            "year",
            "month",
            "day",
            "week",
            "weekday",
            "hour",
        ]
    )
df = df[["date", "Ret"]]

df["Ret_10"] = df["Ret"].rolling(10).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret_50"] = df["Ret"].rolling(50).apply(lambda x: np.prod(1 + x / 100) - 1)

df["Ret_25"] = df["Ret"].rolling(25).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret25"] = df["Ret_25"].shift(-25)
del df["Ret_25"]
df = df.dropna()
```

Next, we need to:

- Define the different sets (validation, train and test), as well as the window size.
- Scale input and outputs using the MinMaxScaler

```
In [ ]: val_split = 0.2
train_split = 0.625
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 30

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")
```

```

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

# Times Series
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)

print("Shape of test data", X_test.shape, y_test.shape)

```

\

3.1. Model Architecture:

The next step is defining the architecture of the model. We will, again, follow the exact same architecture as in Lesson 2, with one exception: we will use Bidirectional LSTMs.

The way to set up a bidirectional layer in Keras / TensorFlow is extremely easy, but you should be aware of the different possibilities and operations that the bidirectional layer conveys. Here you can access the Keras and TensorFlow documentation for bidirectional layers:

- Keras: https://keras.io/api/layers/recurrent_layers/bidirectional/

- TensorFlow:
https://www.tensorflow.org/api_docs/python/tf/keras/layers/Bidirectional#call-arguments_1

```
In [ ]: SEED = 1234

units_lstm = 50
n_dropout = 0.2
act_fun = "relu"

from keras import Sequential
from keras.callbacks import EarlyStopping

model = Sequential()

model.add(
    Bidirectional(
        LSTM(
            units=units_lstm,
            return_sequences=True,
            activation="tanh",
            input_shape=(X_train.shape[1], n_features),
        )
    )
)
model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=False, activation="tan
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=20, activation=act_fun)) # 40
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=10, activation=act_fun)) # 10
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(1))
```

Now we are ready to train the model!

```
In [ ]: hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=0, patience=5, restore_best_weights=
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=val_split,
    epochs=100,
    batch_size=64,
    verbose=2,
    callbacks=[es],
)
```

```
In [ ]: model.summary()
```

One of the main issues related to the bidirectional network is that they have many more parameters, as we can see with a quick check of the model summary (feel free to go back to the unidirectional LSTM case in lesson 2 to see the differences):

A quick note on the model parameters

Where do the 60,400 parameters in `bidirectional_1` come from? Well, it is basically the same computation as in the unidirectional case, but noticing that there are 8 (2×4) equations with parameters now:

$$\begin{equation*} \# \text{ of parameters} = 2 \times 4 \times \left(\left(\text{Hidden Units} + \text{Input size} \right) \times \text{Hidden Units} + \text{Hidden Units} \right) \end{equation*}$$

Applying the previous to `bidirectional_1`:

- Hidden units = 50
- Input size = 100

Hence, number of parameters:

$$\begin{equation*} 2 \times 4 \times \left(\left(50 + 100 \right) \times 50 + 50 \right) = 60,400 \end{equation*}$$

So, what do predictions from the bidirectional LSTM model look like?

```
In [ ]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
```

```

values = values.flatten()

def R2_campbell(y_true, y_predicted, mean_ret):
    y_predicted = y_predicted.reshape((-1,))
    sse = sum((y_true - y_predicted) ** 2)
    tse = sum((y_true - mean_ret) ** 2)
    r2_score = 1 - (sse / tse)

    return r2_score

R2_Campbell = R2_campbell(values, prediction, mean_ret)
print("Out-of-sample R-squared:", R2_Campbell)

# %%
df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred LSTM": prediction, "values": values}
)
df_predictions.head()

```

In this case, it seems that the bidirectional model does a pretty poor job in terms of out-of-sample R-squared. Let's see how it performs in a backtest.

4. Backtest

Finally, we perform a simple backtest of the trading strategy resulting from the predictions delivered by the model. As was the case in Lesson 2, we will basically trade long (buy BTC) whenever the 25-day prediction is positive and short (sell BTC) when negative.

```

In [ ]: df_predictions["Positions"] = df_predictions["Pred LSTM"].apply(np.sign)
df_predictions["Strat_ret"] = (
    df_predictions["Positions"].shift(1) * df_predictions["values"]
)
df_predictions["CumRet"] = (
    df_predictions["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)
df_predictions["bhRet"] = (
    df_predictions["values"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)

Final_Return = np.prod(1 + df_predictions["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df_predictions["values"]) - 1

print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")

ax = plt.gca()
df_predictions.plot(x="Date", y="bhRet", ax=ax)
df_predictions.plot(x="Date", y="CumRet", ax=ax)
plt.show()

```

As one can easily see from the graph above, the backtest based on the bidirectional LSTM prediction model does a terrible job in the period tested (although still better than

a pure buy-and-hold strategy).

Indeed, the bidirectional model does a worse job than the unidirectional one of lesson 2. This is, however, not usually the case, and bidirectional models tend to perform much better than unidirectional ones. Why is that? Well, for starters, bidirectional models contain much more information, more parameters, and more flexibility. So, why do they perform worse in this case? As always, we have obtained these results under a pre-specified set of hyperparameters for a specific architecture. With models with more parameters, greater care is needed to fine-tune those hyperparameters. It is now your turn to play around with these features to see if you can achieve a better out-of-sample prediction power for the bidirectional model!

5. Conclusion

In this lesson, we have extended the LSTM prediction model framework from lesson 2 to consider bidirectional networks. In this example case, we found little improvement (actually an overall worsening) of model performance in the bidirectional case. However, bidirectional LSTMs usually perform better than the unidirectional models.

This has been all for Module 5. In the next module, we will deal with another powerful feature of neural networks: autoencoders. See you there!